

Phase 0 — Mise en route

Pourquoi LangChain ? Comment installer PyTutor en 15 minutes

Cours PyTutor2

Résumé

Ce document est le point d'entrée du cours **PyTutor2**. Il répond d'abord à la question « *pourquoi LangChain ?* », puis te guide pas à pas dans l'installation : Python, PyCharm, environnement virtuel, dépendances (**requirements.txt**), clés API (**.env**), et la configuration centrale du projet (**config.py**). À la fin, tu lances ton premier appel à un modèle Claude en moins d'une commande et tu es prêt(e) pour la Phase 1.

Table des matières

1	Pourquoi ce cours ?	1
1.1	Ce que tu vas construire	1
1.2	Pourquoi LangChain ?	1
1.3	Outils que tu vas utiliser	2
2	Installer Python	2
2.1	Vérifier ta version	2
3	Installer PyCharm	2
3.1	Téléchargement	2
3.2	Configuration de l'interpréteur Python	3
4	Récupérer le projet	3
5	Créer un environnement virtuel	3
5.1	Création	3
5.2	Brancher PyCharm sur le venv	4
6	Installer les dépendances : requirements.txt	4
6.1	Pourquoi ces paquets ?	5
7	Configurer les clés API : .env	5
7.1	Obtenir les clés	5
7.2	Créer le fichier .env	5
8	La configuration centrale : config.py	6
8.1	Trois choses à retenir	7
9	Premier test — Hello, Claude	7
9.1	Que faire si ça plante ?	8
10	Activer LangSmith (recommandé)	8
	Récapitulatif	8

1 Pourquoi ce cours ?

1.1 Ce que tu vas construire

En quatre phases, tu vas assembler un **tuteur Python** qui sait expliquer, chercher dans une documentation, et exécuter du code pour vérifier ses propres réponses :

- **Phase 1 — LangChain LCEL.** Tu apprends à composer des chaînes de traitement avec l'opérateur `| : prompt | modèle | parser`. Tu manipules `ChatPromptTemplate`, sorties typées `Pydantic`, `RunnableParallel`, et streaming token par token.
- **Phase 2 — RAG.** Tu construis un système qui *récupère* des passages de la doc Python officielle avant de répondre. Tu maîtrises l'indexation (embeddings + Chroma), le retriever LCEL, et la comparaison *similarity vs MMR*.
- **Phase 3 — Agent.** Tu fabriques un agent qui décide *lui-même* quels outils appeler : chercher dans la doc, exécuter du code Python, générer un exercice, vérifier la PEP 8. Tu observes la boucle *Réfléchir-Agir-Observer* en direct.

1.2 Pourquoi LangChain ?

Construire une application LLM sans framework, c'est jouable — quelques requêtes HTTP, deux ou trois `f"..."` pour formater les prompts. Mais dès que tu veux :

- Enchaîner plusieurs étapes (récupération, prompt, modèle, parser) sans réinventer le tuyau à chaque fois,
- Brancher un *vector store* pour faire du RAG,
- Donner à un modèle des *outils* qu'il peut appeler en autonomie (agent),
- Tracer chaque appel pour déboguer une chaîne complexe,

... il te faut un **vocabulaire commun** et des **briques composables**. C'est exactement ce que fournit **LangChain 1.x** via *LCEL* (LangChain Expression Language) et son module `langgraph` pour les agents.

***Règle pratique.** LangChain n'est pas un mur magique : c'est un ensemble de **Runnable** qui se composent avec `|`, `.batch`, `.stream`, `.invoke`, et qui s'observent dans *LangSmith*. Une fois ces 4 verbes maîtrisés, tout le reste se déduit.*

1.3 Outils que tu vas utiliser

Outil	Rôle
Python 3.10+	Langage du projet.
PyCharm	IDE confortable (autocomplétion, debug, run).
venv	Environnement Python isolé pour ce projet.
LangChain 1.x	Composition de chaînes LCEL.
Chroma	Base vectorielle locale (Phase 2).
HuggingFace	Modèle d'embeddings local (Phase 2).
Claude (API)	Modèle LLM d'Anthropic (clé API requise).
LangSmith	Observabilité : voir chaque appel LLM dans une UI web (recommandé).

2 Installer Python

2.1 Vérifier ta version

Ouvre un terminal et tape :

```
python3 --version
```

Tu dois voir Python 3.10.x ou plus récent. Si la commande n'existe pas ou affiche une version inférieure, installe une version récente :

- **macOS** : python.org ou via Homebrew : `brew install python@3.12`.
- **Windows** : installateur officiel python.org. *Coche bien* Add Python to PATH pendant l'installation.
- **Linux** : ton gestionnaire de paquets, par exemple `sudo apt install python3.12 python3.12-venv`.

Pourquoi 3.10+ ?

- LangChain 1.x requiert Python 3.10 minimum.
- `match/case` (PEP 634) est utile en Phase 3.
- Les annotations modernes (`X | None` au lieu de `Optional[X]`) marchent partout.

Recommandation : 3.12 est le bon défaut en 2026.

3 Installer PyCharm

PyCharm Community Edition est gratuit et largement suffisant pour ce cours : pas besoin de la version Pro.

3.1 Téléchargement

1. Va sur jetbrains.com/pycharm/download.
2. Choisis **PyCharm Community** (et pas Professional).
3. Lance l'installateur et suis les étapes par défaut.

3.2 Configuration de l'interpréteur Python

À la première ouverture, PyCharm te demande quel interpréteur utiliser. Tu peux :

1. Pointer vers ton Python système (rapide, mais pollue tes installations Python).
2. **Recommandé** : créer un *virtualenv* dédié au projet (voir Section 5).

***Astuce.** Si tu préfères travailler dans un terminal pur (VS Code, Neovim, terminal système...), tu peux. PyCharm est recommandé mais pas obligatoire. Les commandes du cours sont toujours données en ligne de commande.*

4 Récupérer le projet

```
git clone <URL-du-repo>
cd pytutor
```

Si tu n'utilises pas git, télécharge le projet sous forme de .zip et décompresse-le. À ce stade, ton dossier ressemble à :

```
pytutor/
  config.py
  phase1_explain.py
  phase2_rag.py
  phase3_agent.py
```

```
phase1-solutions.py
phase2-solutions.py
phase3-solutions.py
requirements.txt
.env.example      # à copier en .env
Latex/            # les documents PDF du cours
```

5 Créer un environnement virtuel

Un *venv* isole les dépendances du projet du reste de ta machine. C'est obligatoire en pratique pour ne pas casser ton système Python.

5.1 Création

```
cd pytutor
python3 -m venv .venv
```

Un dossier `.venv/` apparaît. Active-le :

— **macOS / Linux :**

```
source .venv/bin/activate
```

— **Windows (PowerShell) :**

```
.venv\Scripts\Activate.ps1
```

— **Windows (cmd.exe) :**

```
.venv\Scripts\activate.bat
```

Une fois activé, ton prompt commence par `(.venv)`. Tape `python --version` pour vérifier que c'est bien le Python du `venv` qui répond.

5.2 Brancher PyCharm sur le `venv`

File → *Settings* (ou *PyCharm* → *Preferences* sur macOS) → *Project : pytutor* → *Python Interpreter* → *Add Interpreter* → *Existing* → sélectionner `pytutor/.venv/bin/python` (ou `.venv\Scripts\python.exe` sur Windows).

6 Installer les dépendances : `requirements.txt`

Le fichier `requirements.txt` liste tout ce dont le projet a besoin :

```
# --- LangChain core (Phase 1, 2 et 3) ---
# Note : LangChain 1.0 (oct. 2025) a refactorisé l'écosystème.
# Le méta-package `langchain` est désormais slim ; les anciennes classes
# `langchain.retrievers.*`, `langchain.chains.*`, etc., sont dans `langchain-classic`.
langchain>=1.0
langchain-classic>=1.0 # MultiQueryRetriever, ContextualCompressionRetriever, ...
langchain-anthropic>=1.0
langgraph>=1.0
```

```
# --- Vector store local (Phase 2) ---
langchain-chroma>=0.1.4
chromadb>=0.5

# --- Embeddings locaux, gratuits, sans clé API supplémentaire (Phase 2) ---
# (~500 Mo à l'installation à cause de torch, mais ça reste local après)
langchain-huggingface>=0.1
sentence-transformers>=3.0

# --- Chargement de documents web (Phase 2) : requests + BeautifulSoup ---
# Remplace WebBaseLoader de langchain-community (sunset en LangChain 1.x).
requests>=2.31
beautifulsoup4>=4.12
lxml>=5.0

# --- Utilitaires ---
python-dotenv>=1.0
pydantic>=2.0
pycodestyle>=2.12 # utilisé par le tool check_pep8 de la Phase 3
```

Installe-les avec :

```
pip install --upgrade pip
pip install -r requirements.txt
```

Combien de temps ? Combien de Mo ?

- Premier pip install : **5 à 10 minutes** selon ta connexion. La plus grosse partie est torch et sentence-transformers (~500 Mo).
- Les lancements suivants utilisent le cache pip : instantané.
- Le venv complet pèse autour de **1.5 Go**. C'est normal pour un environnement ML.

6.1 Pourquoi ces paquets ?

Paquet	À quoi ça sert
langchain	Cœur du framework : Runnable, ChatPromptTemplate, parsers.
langchain-classic	Ancienne API conservée pour quelques retrievers (ex. MultiQueryRetriever).
langchain-anthropic	Wrapper officiel pour Claude.
langgraph	Construire des agents avec une boucle Réfléchir-Agir-Observer.
langchain-chroma	Intégration Chroma pour LangChain.
chromadb	Base vectorielle locale, persistée sur disque.
langchain-huggingface	Wrapper pour les embeddings HuggingFace.
sentence-transformers	Modèle d'embeddings (encode les textes en vecteurs, local, gratuit).
requests, bs4, lxml	Aspirer des pages web pour l'indexation.
python-dotenv	Charger .env automatiquement.
pydantic	Schémas typés (sortie structurée).
pycodestyle	Outil PEP 8 utilisé en Phase 3.

7 Configurer les clés API : .env

Deux services en ligne sont utilisés :

- **Anthropic** (Claude) — *obligatoire*. Le modèle LLM qui répond à toutes tes requêtes.
- **LangSmith** — *fortement recommandé*. Une UI web qui trace chaque appel LLM. Indispensable pour comprendre ce que LangChain fait *vraiment*.

7.1 Obtenir les clés

1. **Anthropic** : crée un compte sur console.anthropic.com, va dans *API keys*, et crée une clé qui commence par `sk-ant-api03-....`.
2. **LangSmith** : crée un compte gratuit sur smith.langchain.com, va dans *Settings* → *API Keys*, et crée une clé qui commence par `lsv2_pt_....`.

Avertissement crédits / coûts

- L'API Anthropic est **payante** mais Claude Haiku est très bon marché. Tu peux suivre tout le cours pour quelques dollars de crédits.
- LangSmith offre une **enveloppe gratuite** largement suffisante pour un usage pédagogique.

7.2 Créer le fichier `.env`

Crée un fichier nommé `.env` à la racine du projet :

```
# --- Anthropic (Claude) - OBLIGATOIRE ---
ANTHROPIC_API_KEY=sk-ant-api03-XXXXXXXXXXXXXXXXXXXXXXXXXXXX

# --- LangSmith (fortement recommandé) ---
# Crée un compte gratuit sur https://smith.langchain.com
# Tu verras chaque appel, chaque token, chaque décision d'agent
# dans une UI web. C'est le meilleur outil de debug.
LANGSMITH_TRACING=true
# LANGSMITH_ENDPOINT non défini -> US par défaut (smith.langchain.com)
# Pour l'EU, mets : LANGSMITH_ENDPOINT=https://eu.api.smith.langchain.com
#                      + une clé créée sur eu.smith.langchain.com
LANGSMITH_API_KEY=lsv2_pt_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
LANGSMITH_PROJECT=PyTutor2

# Désactive le parallélisme des tokenizers HuggingFace pour éviter
# les warnings bruyants en CLI.
TOKENIZERS_PARALLELISM=false
```

Sécurité — Ne JAMAIS commiter `.env`

- Le `.env` contient des clés en clair : il doit rester **strictement local**.
- Vérifie que ton `.gitignore` contient bien la ligne `.env`.
- Si tu commit une clé par erreur, **révoque-la immédiatement** depuis la console Anthropic / LangSmith et régénère-en une nouvelle.

8 La configuration centrale : `config.py`

Toutes les phases du cours importent leur modèle depuis un unique fichier `config.py`. Ça garantit qu'un changement de modèle (par exemple Haiku → Sonnet) se propage à tout le projet *sans* chercher tous les fichiers.

```
1 """Configuration centrale du projet PyTutor.
2
```

```

3  Toutes les phases important depuis ce module pour rester cohérent.
4  """
5  from __future__ import annotations
6
7  import os
8  from dotenv import load_dotenv
9  from langchain_anthropic import ChatAnthropic
10
11 # Charge automatiquement les variables du .env dès l'import
12 load_dotenv()
13
14 # --- Choix du modèle ---
15 # Haiku = rapide et économique, parfait pour itérer toute la journée.
16 # Sonnet = plus intelligent, à activer quand tu veux comparer la qualité.
17 HAIKU = "claude-haiku-4-5-20251001"
18 SONNET = "claude-sonnet-4-6"
19
20 DEFAULT_MODEL = HAIKU
21
22
23 def get_model(
24     model_name: str = DEFAULT_MODEL,
25     temperature: float = 0.0,
26     max_tokens: int = 2048,
27 ) -> ChatAnthropic:
28     """Renvoie une instance configurée de ChatAnthropic.
29
30     Lève une erreur claire si la clé API n'est pas trouvée.
31     """
32     if not os.getenv("ANTHROPIC_API_KEY"):
33         raise RuntimeError(
34             "ANTHROPIC_API_KEY introuvable. "
35             "As-tu créé un fichier .env à partir de .env.example ?"
36         )
37     return ChatAnthropic(
38         model=model_name,
39         temperature=temperature,
40         max_tokens=max_tokens,
41     )

```

8.1 Trois choses à retenir

1. `load_dotenv()` charge `.env` dès l'import. Tu n'as plus besoin de la rappeler ailleurs.
2. `get_model()` est le **point d'entrée unique** pour obtenir un modèle. Tous les exercices font `from config import get_model`.
3. Pour passer en *Sonnet* le temps d'un test : `get_model(SONNET)`. Pour augmenter la créativité : `get_model(temperature=0.7)`. Tout reste local à ton appel.

9 Premier test — *Hello, Claude*

À ce stade, tu as : Python, PyCharm, le venv activé, les dépendances installées, le `.env` rempli, et `config.py` en place. Crée un fichier `test_hello.py` à la racine du projet avec le contenu ci-dessous. C'est un mini-aperçu de *LCEL* : on compose trois briques (`prompt`, `model`, `parser`) avec l'opérateur `|`, puis on `.invoke` la chaîne avec un dict d'entrée.

```

1  """test_hello.py - valide l'installation et donne un avant-goût LCEL."""

```

```

2 from langchain_core.prompts import ChatPromptTemplate
3 from langchain_core.output_parsers import StrOutputParser
4
5 from config import get_model
6
7 # 1. Définir le gabarit de prompt
8 prompt = ChatPromptTemplate.from_template(
9     "Raconte-moi une blague courte sur le thème : {sujet}"
10 )
11 # 2. Initialiser le modèle (nécessite la clé API)
12 model = get_model()
13 # 3. Créer le parser de sortie
14 parser = StrOutputParser()
15 # 4. Assembler la chaîne (LCEL)
16 chain = prompt | model | parser
17 # 5. Exécuter la chaîne
18 reponse = chain.invoke({"sujet": "Cours de mathématiques"})
19 print(reponse)

```

Lance-le :

```
python test_hello.py
```

Si tu vois une blague s'afficher, félicitations : ton setup est valide.

Sortie de l'exécution

```

# Une blague de maths

Un professeur de mathématiques demande à son élève :

-- Pourquoi tu n'as pas fait tes devoirs ?

-- Parce que j'ai essayé de résoudre une équation du second degré,
   mais j'ai perdu le discriminant en chemin !

-- Comment on perd un discriminant ?

-- Ben, c'était un Delta, alors il a dû s'envoler !

```

Note. La sortie change à chaque appel (le modèle est non déterministe). Avec *temperature=0* (le défaut dans *config.py*), tu obtiendras des réponses très similaires d'une exécution à l'autre, mais rarement identiques au caractère près.

9.1 Que faire si ça plante ?

Erreur	Cause probable
ANTHROPIC_API_KEY introuvable	Tu n'as pas créé le <code>.env</code> , ou il n'est pas à la racine du projet.
ModuleNotFoundError: langchain	Le <code>venv</code> n'est pas activé, ou tu as oublié <code>pip install -r requirements.txt</code> .
401 Unauthorized (Anthropic)	Clé invalide ou expirée. Régénère-en une dans la console.
429 Too Many Requests	Tu as épuisé tes crédits ou tu cognes le rate limit. Vérifie ton compte Anthropic.
Compilation très lente	Premier lancement : <code>sentence-transformers</code> télécharge un modèle (~90 Mo). C'est normal, attends.

10 Activer LangSmith (recommandé)

Si tu as bien rempli `LANGSMITH_API_KEY` dans `.env`, chaque appel LLM est désormais tracé sur smith.langchain.com, dans le projet `PyTutor2`.

- Va sur l'UI, ouvre le projet `PyTutor2`.
- Tu vois la liste des derniers *runs*.
- Clique sur un run pour voir : l'arbre d'exécution, le prompt *exactement* envoyé au modèle, les tokens, la latence, le coût.

LangSmith change la donne

Sans LangSmith, déboguer une chaîne LangChain c'est ajouter des `print` partout puis essayer de reconstituer ce qui s'est passé. Avec LangSmith, l'arbre d'exécution est directement visualisé. **Active-le maintenant**, tu gagneras des heures sur toutes les phases qui viennent.

Récapitulatif

Ce qu'il faut retenir pour démarrer :

1. *Python 3.10+ obligatoire (3.12 recommandé).*
2. *PyCharm Community suffit (ou n'importe quel autre IDE).*
3. *Toujours travailler dans un `venv` dédié.*
4. *`pip install -r requirements.txt` une seule fois.*
5. *Créer un `.env` avec les clés Anthropic et LangSmith.*
6. *Toutes les phases importent `get_model()` depuis `config.py` : changer de modèle se fait en une ligne.*
7. *Lancer `python phase1_explain.py` pour valider le setup avant d'attaquer la Phase 1.*

Prêt ? Direction la Phase 1

Dans la **Phase 1**, on entre dans le vif du sujet : `ChatPromptTemplate`, sortie structurée Pydantic, composition LCEL avec `|`, `RunnableParallel`, et streaming. C'est le socle de tout le reste du cours.